

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3.6

UPGRADE TO 4.0

 Copy as Markdown

Upgrade to 4.0

Serverpod 4.0 (Jetstream) adds `serverpod start`, one command that runs your server, database, and Flutter app together with hot reload.

The required steps take about 15 minutes: update the CLI and your dependencies, regenerate code, and create a migration. Plan for more time if breaking changes affect your code, or if you have a Dockerfile or CI workflows to update. The new `serverpod start` workflow, the embedded Postgres, and the agent setup are optional.

Before you start

- You have Flutter 3.44.4 or later. It includes Dart 3.12.2, which Serverpod 4.0 requires. Check with `flutter --version`, and run `flutter upgrade` if your version is older.
- Your project is on the latest Serverpod 3.4.x release.
- Your project compiles and tests pass.
- You've committed your current state to Git so you can roll back if needed.

Update the Serverpod CLI

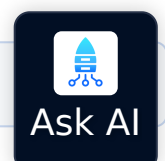
The 3.4 CLI was installed with `dart pub global activate`. Deactivate it before you install the 4.0 CLI:

```
$ dart pub global deactivate serverpod_cli
```

Then install the 4.0 CLI:

```
$ dart install serverpod_cli
```

Verify the version:



```
$ serverpod version
```

Update your project dependencies

Bump every Serverpod package to 4.0 in the `pubspec.yaml` files of `<project>_server`, `<project>_client`, and `<project>_flutter`. Bump them in both `dependencies` and `dev_dependencies`. Pin the exact version rather than a caret range, so the packages stay in sync with the CLI:

```
dependencies:
  serverpod: 4.0.0           # in the server package
  serverpod_client: 4.0.0    # in the client package
  serverpod_flutter: 4.0.0   # in the Flutter package

dev_dependencies:
  serverpod_test: 4.0.0      # in the server package
```

Don't skip `serverpod_test`. It pins `serverpod` to an exact version. If you leave `serverpod_test` on 3.4 in `dev_dependencies`, `dart pub upgrade` fails.

Bump the Dart SDK constraint in the root `pubspec.yaml` and `<project>_server/pubspec.yaml` to the 4.0 minimum:

```
environment:
  sdk: '^3.12.2'
```

If your project uses the new auth module, bump its packages in the same files:

```
dependencies:
  serverpod_auth_idp_server: 4.0.0 # in the server package
  serverpod_auth_idp_client: 4.0.0 # in the client package
  serverpod_auth_idp_flutter: 4.0.0 # in the Flutter package
```

If you use the legacy auth module

The legacy `serverpod_auth` packages ship 4.0 releases. Bump every `serverpod_auth` package your project uses to the same version as Serverpod itself, in the same `pubspec.yaml` files:

```
dependencies:
  serverpod_auth_server: 4.0.0 # in the server package
  serverpod_auth_client: 4.0.0 # in the client package
  serverpod_auth_shared_flutter: 4.0.0 # in the Flutter package
```

The `authenticationKeyManager` parameter on the generated `Client` was removed in 4.0. Assign the key manager to the `authKeyProvider` field instead:

```
client = Client('http://$ipAddress:8080/')
  ..authKeyProvider = FlutterAuthenticationKeyManager()
  ..connectivityMonitor = FlutterConnectivityMonitor();
```

If you use legacy Sign in with Apple, set `appleClientIds` on `AuthConfig`, because Sign in with Apple is disabled until it's set. See [Apple sign-in](#).

The legacy module keeps working on 4.0, so you can upgrade without moving to the new authentication framework. To make that move, finish this upgrade first, then see [Migrate from legacy auth](#).

If you use the new auth module on Android

The `serverpod_auth_core_flutter` package now requires `flutter_secure_storage` 10.0.0 or newer and allows 11.x. Most projects already resolve 10.x and are not affected.

If your Flutter app is still on 9.x, you have two options:

- **Upgrade directly to 11.x.** Version 11 dropped the code that migrates data written by 9.x, so users on Android are signed out and have to sign in again.
- **Upgrade to 10.x first.** Users stay signed in when you upgrade from 9.x to 10.x, and again from 10.x to 11.x, because each step keeps the session. Release a 10.x build, let it reach your users, then move to 11.x.

Projects created with `serverpod create` pin 10.x with a dependency override. To do the same, add this to the Flutter app's `pubspec.yaml`:

```
dependency_overrides:
  flutter_secure_storage: ^10.0.0
```

Version 11 also requires `compileSdk = 37` in `android/app/build.gradle.kts`. That value is higher than the current Flutter default.

Refresh dependencies and generated code

From the project's root folder, refresh dependencies. Projects created with the 3.3+ scaffold use a Dart workspace. A workspace resolves all sub-packages in one command:

```
$ dart pub upgrade
```

If the root `pubspec.yaml` has no `workspace:` block, your project doesn't use a Dart workspace. In that case, run `dart pub upgrade` separately in each sub-package. To adopt workspaces, see Dart's [pub](#)

[workspaces documentation](#).

Before you generate code, rename model files that use a plain `.yaml` or `.yml` extension to `.spy.yaml`, so `serverpod generate` doesn't stop with an error. See [Model files use the .spy.yaml extension](#) for details.

Then refresh the generated server and client code:

```
$ serverpod generate
```

If `serverpod generate` reports `analysis skipped due to invalid Dart syntax` for an endpoint or future call file, that file doesn't compile against 4.0. The error usually means the file still uses an API that changed in 4.0. Fix the file with the matching section under [Other breaking changes](#), then run `serverpod generate` again.

Other breaking changes

These changes can break code that worked on 3.4. Find the ones that apply to your project, then read only those sections. Changes to sign-in and tokens are in [Authentication changes](#).

Change	Affects you if
Model files use the .spy.yaml extension	Your model files end in <code>.yaml</code> or <code>.yml</code> instead of <code>.spy.yaml</code> .
Client exceptions are a sealed hierarchy	Your app catches <code>ServerpodClientException</code> or checks <code>statusCode == -1</code> .
Database exceptions are typed	Your server catches <code>DatabaseInsertRowException</code> or another row exception.
Future calls run at least once	You use future calls.
Removed deprecated APIs	You use the string-based future call methods, <code>orderDescending</code> , <code>ignoreEndpoint</code> , <code>SerializationManagerServer</code> , the old web widget classes, or <code>--mini</code> .
Message central delivers globally by default	You call <code>session.messages.postMessage</code> with Redis enabled, or pass <code>global: true</code> .

Change	Affects you if
File storage APIs are renamed	Your server uses <code>session.storage</code> , subclasses <code>CloudStorage</code> , or serves public files from native Google Cloud Storage.
Google sign-in on the web uses the OAuth2 redirect flow	Your Flutter web app uses Google sign-in from <code>serverpod_auth_idp_flutter</code> .
Sign-in buttons share one set of style enums	You set style arguments on sign-in buttons from <code>serverpod_auth_idp_flutter</code> .
Legacy streaming endpoints are removed	Your endpoints use <code>StreamingSession</code> .
Insights database endpoints are disabled by default	Your own tooling calls the Insights database endpoints.
The <code>columnOverride</code> experimental feature is removed	You pass <code>--experimental-features columnOverride</code> .
<code>Session.close()</code> returns <code>Future<void></code>	Your code uses the value that <code>session.close()</code> returns.
Unused email exceptions are removed	Your app catches <code>EmailAccountRequestAlreadyExistsException</code> or <code>EmailPasswordResetAccountNotFoundException</code> .

Model files use the `.spy.yaml` extension

In 4.0, model files must use the `.spy.yaml` extension. The `.spy.yml` and `.spy` extensions are also accepted. Serverpod ignores files in `lib/src/models` or `lib/src/protocol` that have a plain `.yaml` or `.yml` extension. The `serverpod generate` and `serverpod start` commands stop with an error that lists those files:

```
Model files must use the .spy.yaml extension. The following files are ignored:
lib/src/models/company.yaml
Rename the files to use the .spy.yaml extension and run the command again.
```

Rename the files and run `serverpod generate` again. The contents do not change.

Client exceptions are a sealed hierarchy

The `ServerpodClientException` class is sealed and carries only `message`. When the app can't reach the server, the call throws `ServerpodClientNetworkException`. When the server returns an error response, the call throws a subclass of the sealed `ServerpodClientHttpException`, which owns `statusCode`. Code that checks `statusCode == -1` for connection failures no longer compiles. See [Error handling and exceptions](#).

Database exceptions are typed

The `DatabaseInsertRowException`, `DatabaseUpdateRowException`, `DatabaseDeleteRowException`, and `DatabaseUpsertRowException` classes are replaced by `DatabaseUnexpectedResultException`. Constraint failures throw `DatabaseUniqueViolationException` or `DatabaseForeignKeyViolationException`. SQLite lock failures throw `SqliteDatabaseLockedException`. All three are subclasses of `DatabaseQueryException`. See [Database exceptions](#).

Future calls run at least once

In 3.4, Serverpod removed a future call from the database before running it, so a crash mid-run lost the call. In 4.0, Serverpod removes the call only after it finishes. If a crash interrupts the call, the call runs again, possibly on another server instance. Make each future call safe to run more than once, for example by checking whether an email was already sent before sending it. See [Execution guarantees](#).

Removed deprecated APIs

- The future call methods on `Serverpod` are removed: `registerFutureCall`, `futureCallWithDelay`, `futureCallAtTime`, and `cancelFutureCall`. The `invoke` method you overrode on `FutureCall` is removed too. Calls already scheduled under an old string name match no generated call, so Serverpod doesn't run them and reports them as [broken future calls](#). To port a string-registered call:
 - Replace the `invoke` override with a public `Future<void>` method that takes a `Session` as its first parameter. See [Define a future call](#).
 - Delete the `registerFutureCall` line from `server.dart`, and run `serverpod generate`.
 - Schedule with `pod.futureCalls.callWithDelay` or `pod.futureCalls.callAtTime`, and cancel with `pod.futureCalls.cancel`. See [Schedule a call](#) and [Cancel scheduled calls](#).
- The `orderDescending` parameter on ORM methods is removed. You can no longer construct `Order` directly. Use `column.asc()` and `column.desc()`. See [Sorting](#).
- The `ignoreEndpoint` annotation is removed. Use `@doNotGenerate`. See [Exclude an endpoint from generation](#).
- The `SerializationManagerServer` class is removed. Use the generated `Protocol` class instead. It now extends `DatabaseSerializationManager` from `serverpod_database`.
- The deprecated web-server widget aliases are removed. Rename them:
 - `AbstractWidget` to `WebWidget`

- `Widget` to `TemplateWidget`
- `WidgetList` to `ListWidget`
- `WidgetJson` to `JsonWidget`
- `WidgetRedirect` to `RedirectWidget`
- If a widget extends `WebWidget` directly, implement the new abstract `String render({String? Function(String)? onMissingVariable})` method. The `WidgetRoute` class builds the response from `render` instead of `toString`. Widgets that extend `TemplateWidget`, `ListWidget`, `JsonWidget`, or `RedirectWidget` inherit `render`. The `WidgetRoute.build` method now returns `Future<WebWidget?>`. A `null` return yields a 404. See [Server-side HTML](#).
- The `RouteStaticDirectory` and `PathCacheMaxAge` classes are removed. Serve directories with `StaticRoute.directory`, and set cache headers with its `cacheControlFactory` parameter. See [Static files](#).
- The `--mini` flag on `serverpod create` is removed. To create a project without a database, deselect **Database (recommended)** on the setup screen, or pass `--no-interactive --no-database`. To create a project without a Flutter app, use `--template server`.

Message central delivers globally by default

Calls to `session.messages.postMessage` now default to `MessageScope.auto`. When Redis is enabled, the message goes through Redis to every server instance. Otherwise, the message stays local. If your code relied on local-only delivery, pass `scope: MessageScope.local`. The `global: true` argument is removed, so replace it with `scope: MessageScope.global`. See [Message scope](#).

File storage APIs are renamed

Several `session.storage` methods are renamed in 4.0. The methods that return a single file or URL now throw instead of returning `null`. Update your server code as follows:

3.4	4.0
<code>getPublicUrl</code> returns <code>Uri?</code>	<code>publicDownloadUrl</code> returns <code>Uri</code>
<code>getPublicUrls</code>	<code>publicDownloadUrls</code>
<code>retrieveFile</code> returns <code>ByteData?</code>	<code>retrieveFile</code> returns <code>ByteData</code>
<code>storeFile</code> takes <code>expiration</code> and <code>preventOverwrite</code>	<code>storeFile</code> takes <code>options: StoreFileOptions(...)</code>
<code>createDirectFileUploadDescription</code> returns <code>String?</code> and takes <code>expirationDuration</code> ,	<code>createUploadDescription</code> returns <code>String</code> and takes

3.4	4.0
<code>maxFileSize</code> , <code>contentLength</code> , and <code>preventOverwrite</code>	<code>options: UploadOptions(...)</code>
<code>verifyDirectFileUpload</code>	<code>verifyUpload</code>

Catch the new exceptions where your code checked for `null`:

- **No file at the path:** the `retrieveFile` and `publicDownloadUrl` methods throw `CloudStorageFileNotFoundException`. The new `statFile` and `temporaryDownloadUrl` methods throw it too.
- **No public URLs:** the `publicDownloadUrl` method throws `CloudStorageUnsupportedOperationException` when the storage doesn't serve public URLs, for example the default `private` storage.

Both exceptions extend `CloudStorageException`. The `publicDownloadUrls` method still returns `null` for each path that fails.

If you subclass `CloudStorage`, make these changes:

- Override the renamed methods under their new names, and add `statFile` and `temporaryDownloadUrl`.
- Throw `CloudStorageFileNotFoundException` from `retrieveFile`, `publicDownloadUrl`, and `temporaryDownloadUrl` when a file is missing. Throw it from `statFile` too, because the `CloudStorage` base class now implements `fileExists` by calling `statFile`.
- Drop the `verified` argument from `storeFile`, and take a `StoreFileOptions` parameter in place of `expiration`.
- Take an `UploadOptions` parameter in `createUploadDescription`, in place of `expirationDuration` and `maxFileSize`. Return a `BinaryUploadDescription` or a `MultipartUploadDescription` instead of a `String`.
- Move the logic from `storeFileWithOptions` and `createDirectFileUploadDescriptionWithOptions` into `storeFile` and `createUploadDescription`, because the `CloudStorageWithOptions` mixin and the `CloudStorageOptions` class are removed.

If the app uploads or downloads through the built-in `/serverpod_cloud_storage` URL, extend `DatabaseCloudStorage`. That URL rejects other storages. If the subclass keeps bytes outside the database, also override `storeUnverifiedFile`, `retrieveFileWithStat`, and `fileExists`. The URL calls the first two. The `DatabaseCloudStorage` version of `fileExists` checks the database instead of calling `statFile`. See [Store files on local disk](#).

If you serve public files from `NativeGoogleCloudStorage`, make the bucket itself publicly readable, because Serverpod 4.0 no longer makes each uploaded file public. For example, turn on uniform bucket-level access and grant `allUsers` the Storage Object Viewer role.

App code that uses `FileUploader` doesn't change. See [File uploads](#) and [Custom cloud storage](#).

Google sign-in on the web uses the OAuth2 redirect flow

If your Flutter web app uses Google sign-in from `serverpod_auth_idp_flutter`, sign-in now redirects the browser to a callback page. The legacy `serverpod_auth` module is unaffected.

When Serverpod serves your Flutter web app, update the setup:

1. Register a `FlutterWebAuth2CallbackRoute` on the web server. Its full URL is the callback URL, for example `http://localhost:8082/auth/callback`.
2. Pass the callback URL to `initializeGoogleSignIn` as `redirectUri`. Also pass the client ID as `clientId`, or set `GOOGLE_CLIENT_ID` with `--dart-define`. The web flow doesn't read the `google-signin-client-id` meta tag in `web/index.html`.
3. Add the full callback URL in two places: under **Authorized redirect URIs** on the server OAuth client, and in `redirect_uris` in the `googleClientSecret` entry of `passwords.yaml`. In 3.4, both places held the bare origin, for example `http://localhost:8082`.

See [Google web setup](#).

If the app runs on its own origin, for example with `flutter run -d chrome`, skip step 1, because the browser blocks the route's callback across origins. Place an `auth.html` file in the app's `web/` folder instead. In steps 2 and 3, use that file's URL, for example `http://localhost:49660/auth.html`. See [Separately-hosted Flutter web](#).

Sign-in buttons share one set of style enums

If you set style arguments on a sign-in button from `serverpod_auth_idp_flutter` or `serverpod_auth_idp_flutter_facebook`, update them. The legacy `serverpod_auth` module is unaffected.

- **All providers:** use the shared `SignInButtonSize`, `SignInButtonShape`, `SignInButtonLogoAlignment`, and `SignInButtonTextVariant` enums. The per-provider style types, such as `GSIButtonSize` and `AppleSignInStyle`, are removed.
- **Apple and Facebook:** pass `text` instead of `type`.
- **GitHub, Google, and Microsoft:** remove the `type` argument.
- **Google:** pass a `GoogleButtonStyle` as `style` instead of `theme`, and remove `getButtonText`, `locale`, and `buttonWrapper`.

To style every button inside `SignInWidget` at once, pass a `SignInButtonStyle` as `buttonStyle`. See [Styling the buttons](#).

► Removed style types by provider

Legacy streaming endpoints are removed

Serverpod's legacy streaming endpoints API was deprecated in 3.0 and is removed in 4.0. Endpoints that use the `StreamingSession` type no longer compile. The related server and client methods are gone too, for example `streamOpened`, `streamClosed`, `handleStreamMessage`, `sendStreamMessage`, `getUserObject`, `setUserObject`, and `openStreamingConnection`.

Port code that uses the legacy API to [streaming methods](#). With streaming methods, the endpoint declares `Stream` parameters and return types, and Serverpod manages the connection. State that used to live in a user object becomes a local variable in the streaming method. The method stays alive as long as the stream is open. The old API stays documented in [Streaming endpoints](#) while you port.

Insights database endpoints are disabled by default

In 4.0, the Insights server endpoints that give direct database access are disabled by default:

`fetchDatabaseBulkData`, `runQueries`, `getDatabaseRowCount`, and `executeSql`. They throw an `AccessDeniedException` until you enable them. The Insights app doesn't use these endpoints, so most projects need no change.

If you have custom tooling that calls them through the `serverpod_service_client` package, opt in per environment. Set `enableDatabaseAccess` in the `insightsServer` block of the config file, or set the `SERVERPOD_INSIGHTS_SERVER_ENABLE_DATABASE_ACCESS` environment variable:

```
insightsServer:
  port: 8081
  publicHost: localhost
  publicPort: 8081
  publicScheme: http
  enableDatabaseAccess: true
```

The `hotReload`, `getOpenSessionLog`, and `shutdown` Insights methods are removed. See [Insights](#) for details.

The `columnOverride` experimental feature is removed

The `column` keyword no longer needs an experimental feature, so `columnOverride` is removed. Remove `--experimental-features columnOverride` from your commands and scripts, because the flag stops the CLI with a usage error that reports `"columnOverride" is not in all|databaseSync`. Serverpod

ignores a `columnOverride` entry under `experimental_features` in `<project>_server/config/generator.yaml`, so you can delete the entry. See [Column name override](#).

`Session.close()` returns `Future<void>`

The `Session.close()` method returns `Future<void>` instead of `Future<int?>`. It no longer returns an ID when Serverpod logs the session to the database. Code that uses the returned value no longer compiles. A plain `await session.close();` still works.

Unused email exceptions are removed

The email identity provider's `EmailAccountRequestAlreadyExistsException` and `EmailPasswordResetAccountNotFoundException` classes are removed. Nothing threw them in 3.4, so the app sees the same responses as before. Remove any `catch` clause or `case` that names them.

Authentication changes

Version 4.0 changes a few authentication behaviors that can affect existing apps:

- **The `?auth=` query parameter is no longer accepted.** HTTP calls authenticate through the `Authorization` header, or an auth cookie on the web. Streaming connections authenticate when the stream opens. Upgrade any client from before 4.0 that relied on the query parameter.
- **Signing in on top of another account is rejected.** When a token is issued for a different user from an already-authenticated session, Serverpod throws a `SignInWhileAuthenticatedException` on every platform. Users must sign out before switching accounts. Server code that creates tokens on behalf of another user calls the token manager's `createToken` instead, for example in an admin flow. The `createToken` method skips the sign-in policy and returns the secrets in the response body.
- **Method streams close when the signed-in user changes.** On sign-in and sign-out, open method streams close gracefully on all platforms. Their subscriptions receive `onDone` without an error. New streams connect with the current identity. A token refresh for the same identity keeps streams running.
- **If you use Firebase sign-in, unverified emails are accepted by default.** The default `firebaseAccountDetailsValidation` no longer rejects accounts whose email is not verified. To keep the 3.4 behavior, pass `firebaseAccountDetailsValidation: FirebaseAuthConfig.requireVerifiedEmail`. The app then receives a `FirebaseEmailNotVerifiedException` for those accounts. In 3.4, the same case surfaced as a `FirebaseIdTokenVerificationException`, so update error handling that checks for the old exception. See [Custom account validation](#).
- **If you enable cookie auth for the web, list every browser origin.** The new `authCookie` configuration requires you to list every browser origin in `allowedOrigins`. A browser on an origin that isn't listed loses cross-origin access, including to public endpoints. See [web authentication](#).
- **If you wrote a custom token manager, extend the base class.** The `TokenIssuer` and `TokenManager` classes are now base classes. Implement `createToken` to create the token itself, and

leave `issueToken` alone. The `issueToken` method is non-virtual. It applies the sign-in policy and cookie delivery for every token type.

- **If you wrote a custom identity provider, implement `IdentityProvider`.** The `IdentityProviderBuilder` class now requires a provider that implements `IdentityProvider`. A provider class written for 3.4 must add a `method` getter and a `mergeAuthUsers` method. Replace any `static const String method` with the getter, because a class can't declare a static and an instance member with the same name. Give each provider a unique, non-empty `method`, or Serverpod throws a `StateError` when you register the provider. See [the provider class](#).

Generate the 4.0 migration

Version 4.0 adds a few internal Serverpod tables and updates some indexes to speed up logs in Insights. The migration is built from your generated code, so run `serverpod generate` first if you skipped it. Then create the migration:

```
$ serverpod create-migration --tag "upgrade-4-0"
```

The command writes a new migration to `<project>_server/migrations/`. When you [start the server](#), the `serverpod start` command applies the migration. If you run the server yourself instead, start it once from the server package with `dart run bin/main.dart --apply-migrations`. See [Run the server directly](#).

NOTE

If you use the authentication module, the migration warns about a small change to the table that stores rate-limit attempts. Add `--force` to proceed. The change is safe: accounts, sessions, and other auth data are untouched. The only side effect is that current attempt counters reset to zero.

Production deployment notes

Update the server build

If you compile your server with `dart compile exe`, for example in the `Dockerfile` in `<project>_server`, switch to `dart build cli`. The 4.0 server has native build hooks that `dart compile exe` doesn't support. Instead of a single static binary, the new command produces a bundle: an executable plus its native libraries.

If you deploy with the Dockerfile, copy the updated one from the [4.0 framework template](#) or a fresh 4.0 project's `<project>_server/Dockerfile`. Compared with the 3.4 Dockerfile, the new one:

- Builds from the project root instead of the server directory.

- Copies the bundle directory.
- Points `ENTRYPOINT` at the bundled binary.
- Uses the `dart:3.12.2` base image or newer.

Update the GitHub Actions workflows

If your project still has the GitHub Actions workflows that 3.4 created, replace them. The 3.4 workflows are `analyze.yaml`, `format.yaml`, and `tests.yaml` in `.github/workflows/`. They set up an SDK older than the Dart 3.12.2 that 4.0 requires, so they fail after the upgrade. The `tests.yaml` workflow also installs the 3.4 CLI. Copy the 4.0 versions from the [framework templates](#) and fill in their placeholders:

- `projectname`: your project name.
- `CLI_VERSION`: `4.0.0`.
- `DB_TEST_PASSWORD` and `REDIS_TEST_PASSWORD`: the `test` passwords from `<project>_server/config/passwords.yaml`.

Adopt the new development workflow (optional)

The `serverpod start` command runs your server, your Flutter app, and your database in one terminal, and hot reloads your code when you save. It replaces running `docker compose up`, `dart bin/main.dart`, and `flutter run` separately. See [Running your server](#) for the full workflow.

Before you run it, choose how to handle the database.

Choose your data store

Both options work with `serverpod start`. For the fewest changes, keep your Docker Postgres.

Keep your Docker Postgres (easiest upgrade)

If you develop against a Docker Postgres on 3.4, you can keep it without changing your config.

The `serverpod start` command runs `docker compose up -d` when all of these are true:

- Your server package has a Docker Compose file.
- Your config points at a Postgres on `localhost` with no `dataPath`.
- The services aren't running.

On exit, the command runs `docker compose stop` for the services it started. It leaves the other services running.

New 4.0 projects use the `ghcr.io/serverpod/postgres:16` image, which bundles pgvector and PostGIS. Your existing `pgvector/pgvector:pg16` image keeps working. Switch to the new image when

you need PostGIS. See [Upgrade to PostGIS](#).

Switch to the embedded Postgres (recommended for new development)

The embedded Postgres runs as a child process of your server and uses the same Postgres dialect as production. It needs no Docker. It connects over a Unix domain socket, so it doesn't compete for TCP ports. You can reset it by deleting its data directory. See [Embedded PostgreSQL](#).

To switch, add `dataPath` to the database section of `<project>_server/config/development.yaml` and `<project>_server/config/test.yaml`:

```
database:
  host: localhost
  port: 8090
  name: <project>
  user: postgres
  dataPath: .serverpod/development/pgdata
```

For `test.yaml`, use a separate directory, as new projects do: `dataPath: .serverpod/test/pgdata`.

The data directory holds a complete database, so keep it out of version control. Upgrading doesn't add the ignore rule that new projects get, so add this line to `<project>_server/.gitignore`:

```
.serverpod/
```

Once `dataPath` is set, `serverpod start` uses the embedded Postgres automatically.

Don't add `dataPath` to `production.yaml` or `staging.yaml`. In production, use a managed Postgres, such as a [Serverpod Cloud](#) database, Cloud SQL, or RDS.

The server reaches the embedded Postgres over the socket, but database tools like `psql` connect over TCP. To connect a tool, stop the server and run `serverpod database start`. See [Connect a database tool](#).

NOTE

With `dataPath` set, the server connects to the embedded Postgres even if you pass `--docker`. The `--docker` flag only controls whether `serverpod start` brings up the Docker Compose stack. Your Docker volume stays intact, so remove `dataPath` to go back to the Docker Postgres.

Start the server

From your project's root folder, run:

```
$ serverpod start
```

NOTE

If you ran `serverpod start` before upgrading, delete the `<project>_server/.dart_tool/serverpod` folder before you run this command. It holds the server that `serverpod start` compiled earlier, and an old copy can stop the upgraded server at startup. The next run rebuilds it.

On the first run, the command compiles the native build hooks, which can take about 30 seconds. It also applies the migration you generated above. Then the server starts and watches your project. When you save a file, the command hot reloads the code.

The command also launches your `<project>_flutter` app when that package exists. If the server's `pubspec.yaml` has a `serverpod: flutter_apps:` section, the command instead launches the apps in that section that set `auto_launch: true`.

Debug in VS Code

Your upgraded project keeps its 3.4 `launch.json`, which runs `bin/main.dart`. Projects created with 4.0 have a `launch.json` that attaches the debugger to the processes that `serverpod start` runs. Each configuration in that new file first runs the `serverpod_start` task from `tasks.json`. To copy both files into your project:

1. In an empty folder outside your project, create a throwaway 4.0 project with the same name as yours. Add `--template server` if your project has no Flutter app.

```
$ serverpod create <project>
```

2. Copy `tasks.json` and `launch.json` from the throwaway project's `.vscode` folder into your project's `.vscode` folder, replacing the old files.
3. If `tasks.json` sets `SERVERPOD_PASSWORD_database`, replace its value with the `database` password under `development` in `<project>_server/config/passwords.yaml`. The environment variable overrides `passwords.yaml`. If you leave the throwaway project's password in place, the server gets the wrong database password.

Set up the agent workflow (optional)

Version 4.0 ships AI agent skills and MCP servers for editors like Claude Code and Cursor. They let your agent build, run, and inspect your server. The `serverpod create` command sets them up in a new project.

In an upgraded project, install the skills and register the MCP servers by hand.

⚠ WARNING

Don't use `serverpod create .` to add the agent setup to an existing project. The command writes its template files over yours without checking whether they exist. Your `server.dart` and config files lose their database, Redis, authentication, and web server setup. Your `passwords.yaml` gets new random passwords and secrets. Git can't restore the old file, because the default `.gitignore` excludes it.

► Everything the command can overwrite

Install the agent skills

Install the skills tool:

```
$ dart install skills
```

Then, from your project's root folder, pull the skills for your editor:

```
$ skills get --ide <editor>
```

Replace `<editor>` with the editor you use: `antigravity`, `claude`, `cline`, `codex`, `copilot`, `cursor`, or `opencode`. For VS Code, or to install the skills in the `.agents` folder, use `generic`.

Register the MCP servers

Your agent uses two MCP servers: **Serverpod** and **Dart**. Register them in your editor's config file, at a path relative to your project's root folder. For Claude, create `.mcp.json`:

```
{
  "mcpServers": {
    "serverpod": {
      "command": "serverpod",
      "args": ["mcp-server", "--server-dir", "<project>_server"]
    },
    "dart": {
      "command": "dart",
      "args": ["mcp-server"]
    }
  }
}
```

Replace `<project>_server` with the path to your server package, relative to the project root.

Each editor reads the servers from its own file:

Editor	Config file	Format
Claude	<code>.mcp.json</code>	The sample above.
Cursor	<code>.cursor/mcp.json</code>	The sample above.
VS Code	<code>.vscode/mcp.json</code>	The sample above, with the entries under a <code>servers</code> key instead of <code>mcpServers</code> .
Antigravity	<code>.agents/plugins/serverpod-local/mcp_config.json</code>	The sample above, with the Dart entry named <code>dart-mcp-server</code> . Antigravity also needs a <code>plugin.json</code> file in the same folder, with <code>"name"</code> set to <code>"serverpod-local"</code> .
Codex	<code>.codex/config.toml</code>	Codex's TOML format. Pass <code>--force-roots-fallback</code> to <code>dart mcp-server</code> .
OpenCode	<code>opencode.json</code>	OpenCode's <code>mcp</code> format.

In the Codex and OpenCode formats, register the same two commands as the sample: `serverpod mcp-server --server-dir <project>_server` and `dart mcp-server`.

If you are using Cursor, enable the **Serverpod** and **Dart** MCP servers in your project settings (*Cursor Settings* > *Tools & MCPs*).

What's new in 4.0

- `serverpod start` **terminal UI**: hot reload on save. Press **R** to hot restart, **M** to create and apply a migration, or **P** to create and apply a repair migration.
- Simplified server initialization**: the generated `Serverpod` class comes with `Protocol` and `Endpoints` already set up, so `server.dart` needs only `Serverpod(args)`. Projects that keep their existing imports can stay on `Serverpod(args, Protocol(), Endpoints())`.

- **Flutter app launching** from `serverpod start`, so the Flutter app runs alongside the server in the same terminal UI.
- **AI agent skills and MCP servers** set up during `serverpod create`. Existing projects can [add them manually](#).
- **Embedded Postgres**: develop without Docker by setting `dataPath`.
- **SQLite database support** as an alternative dialect to Postgres.
- **Client-side database generation** for the Flutter app.
- `jsonb` **column support** with GIN index operator classes, and `dynamic` **fields** on models and endpoints.
- `unique` **keyword** for simpler unique indexes in model files.
- `upsert` and `upsertRow` on the ORM, and `asc()` / `desc()` convenience methods on orderable columns.
- **Recurring future calls** with `callRecurring`, on a fixed interval or a cron schedule. See [Recurring tasks](#).
- **OAuth2 PKCE Flutter web redirect** for sign-in flows.
- **Account merging** in the auth module, so a user can link a second sign-in method to an existing account. See [Merging accounts](#).
- **httpOnly cookie authentication for the web**, which keeps browser sign-in tokens out of storage that JavaScript can read. See [web authentication](#).
- **Health endpoints** on the built-in web server.
- **IDE and agent selection** in `serverpod create`.

Troubleshooting

Port conflicts on startup

If you run more than one Serverpod server on the same machine, the servers can conflict on the default ports: 8080 for the main server and 8090 for the database. Stop the other server, or run on different ports.

The server stops because the database doesn't match

After the upgrade, `serverpod start` can report that the latest migration is applied and then stop the server with these warnings:

```
WARNING: The database does not match the target database:
WARNING: Database does not match target state.
Server stopped (exitCode: 1).
```

Stop `serverpod start`, delete the `<project>_server/.dart_tool/serverpod` folder, and run `serverpod start` again. If the warnings remain, create and apply the migration from [Generate the 4.0 migration](#).

Agent skills or MCP servers aren't picked up after setup

Run `skills get --ide <editor>` again from the project's root folder. Then check that your editor's MCP config file is at the path listed in [Register the MCP servers](#). That section also covers the extra steps some editors need, such as enabling the servers in Cursor.

Still stuck?

If something here didn't go as expected, reach out on the [community page](#).

Related

- [Migrations](#): how Serverpod's migration system works under the hood.
- [Build your first app](#): the hands-on tour of the 4.0 workflow, if you want to see `serverpod start` in a project built from scratch.